

CE-1103 — Extraclase 2

Benchmark y Visualización de Estructuras de Datos

Algoritmos y Estructuras de Datos I
Instituto Tecnológico de Costa Rica

19 de mayo de 2026

Resumen

Este informe documenta la implementación y evaluación experimental de seis estructuras de datos: Árbol Binario de Búsqueda (BST), Árbol AVL, Árbol Splay, Árbol Rojo-Negro, Arreglo dinámico y Lista Enlazada Simple. Se desarrolló una aplicación Java con interfaz gráfica que permite medir y comparar tiempos de ejecución y conteos de comparaciones de clave para operaciones de inserción, búsqueda y borrado. Los resultados se presentan en forma de tablas comparativas que vinculan los datos empíricos con la teoría de complejidad asintótica.

Palabras clave: Estructuras de datos, Benchmark, Complejidad asintótica, Árboles balanceados, Visualización.

1. Introducción

1.1. Contexto y Motivación

El desempeño de una aplicación depende en gran medida de la elección de las estructuras de datos subyacentes. Mientras que el análisis teórico nos proporciona complejidades asintóticas (O -grande), es fundamental validar estas predicciones mediante experimentos prácticos sobre datos reales.

La Extraclase 1 de este curso proporcionó la base teórica de las principales familias de árboles binarios de búsqueda: BST simples, árboles balanceados por altura (AVL), árboles con reorganización adaptativa (Splay), y estructuras con invariantes de color (Rojo-Negro). En esta Extraclase 2, implementamos estas estructuras desde cero y las comparamos experimentalmente.

1.2. Objetivos

1. Implementar correctamente seis estructuras de datos con conteo de operaciones.
2. Ejecutar un benchmark reproducible que mida tiempos y comparaciones.
3. Visualizar la construcción paso a paso de cada estructura.
4. Analizar los datos experimentales frente a la teoría de complejidad.
5. Extraer conclusiones sobre la eficiencia relativa de cada estructura.

2. Diseño de la Aplicación

2.1. Arquitectura

La aplicación está dividida en módulos:

- **Estructuras de datos** (`src/structures/`): BST, AVLTree, SplayTree, RedBlackTree, CustomArray, CustomLinkedList.
- **Motor de benchmark** (`BenchmarkEngine`): Orquesta W rondas de warmup + R rondas medidas.
- **Interfaz gráfica** (`BenchmarkFrame`): Panel de configuración, tabla de resultados, controles.
- **Visualización** (`TreeVisualizerPanel`, `SequencePanel`): Dibuja árboles y secuencia paso a paso.
- **Exportación** (`CSVExporter`): Guarda resultados en CSV.

2.2. Protocolo de Medición

Entrada:

- N : cantidad de claves a insertar
- `seed`: generador pseudo-aleatorio para reproducibilidad
- W : rondas de warmup (no contadas, calientan la JVM)
- R : rondas medidas (se promedian)
- Modo de búsqueda: automático (generar aleatoriamente) o manual (usuario proporciona)

Fases por ronda:

1. **Inserción:** Se insertan todas las N claves en el orden generado por la semilla.
2. **Búsqueda:** Se realizan búsquedas sin reconstruir ninguna estructura entre consultas.
3. **Borrado:** Se borran todas las claves hasta vaciar (excepto Red-Black, que es N/A).

Métricas capturadas:

- Tiempo (nanosegundos) por fase.
- Número de comparaciones de clave por fase.
- Altura (para árboles) o tamaño (para lineales).

3. Algoritmos Implementados

3.1. Árbol Binario de Búsqueda (BST)

Invariante: Para todo nodo n , todas las claves en el subárbol izquierdo son menores que n , y todas las del derecho son mayores.

Complejidad:

- Inserción / Búsqueda / Borrado: $O(\log n)$ (mejor caso), $O(n)$ (peor caso, lista degenerada).

Implementación destacada: Conteo de comparaciones en búsqueda y descenso. Borrado mediante sucesor in-order.

3.2. Árbol AVL

Invariante: Para todo nodo, el factor de balance (altura subárbol izq. - altura subárbol der.) está en $[-1, +1]$.

Operaciones de rebalanceo: Rotaciones simples y dobles al insertar o borrar.

Complejidad garantizada: $O(\log n)$ inserción, búsqueda, borrado. Altura $\leq 1,44 \log_2 n$.

3.3. Árbol Splay

Invariante: Ninguna. El árbol se reorganiza mediante operaciones *splay* tras cada acceso.

Heurística: Cada búsqueda o inserción mueve el nodo accedido a la raíz mediante rotaciones. Tiende a optimizar accesos frecuentes.

Complejidad amortizada: $O(\log n)$ en promedio. Operaciones individuales pueden ser $O(n)$.

3.4. Árbol Rojo-Negro (Red-Black)

Invariantes:

1. Cada nodo es rojo o negro.
2. La raíz es negra.
3. Los nodos rojos tienen hijos negros.
4. Todos los caminos desde un nodo a sus hojas nulas contienen el mismo número de nodos negros.

Complejidad garantizada: $O(\log n)$ inserción y búsqueda. Borrado no se mide en este proyecto (N/A).

Nota especial: Solo inserción y búsqueda son medidas. El borrado es teórico pero no se ejecuta en el benchmark.

3.5. Estructura Lineal: Array Dinámico

Inserción: Append en $O(1)$ amortizado (redimensionamiento ocasional).

Búsqueda / Borrado: $O(n)$ lineal.

3.6. Estructura Lineal: Lista Enlazada Simple

Inserción / Borrado: $O(n)$ (búsqueda del punto).

Búsqueda: $O(n)$ lineal.

4. Capturas de Pantalla de la Aplicación

4.1. Panel de Configuración

La Figura 1 muestra el panel izquierdo de la aplicación con todos los parámetros configurables. El usuario puede ajustar N , semilla, W , R , seleccionar modo de búsqueda (automático o manual), y elegir qué estructuras medir.

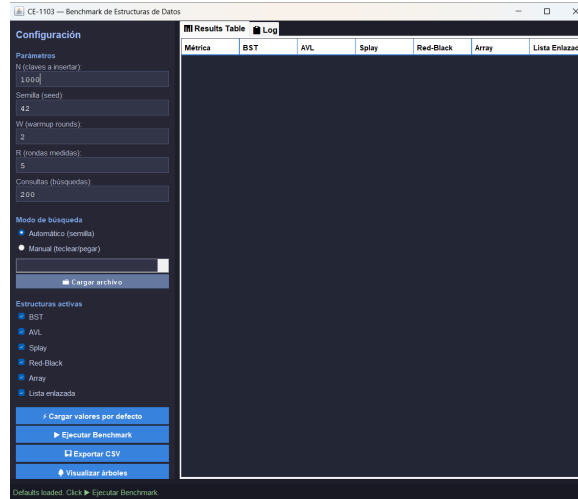


Figura 1: Panel de configuración con parámetros de benchmark ($N=1000$, $seed=42$, $W=2$, $R=5$).

4.2. Tabla Comparativa de Resultados

La Figura 2 muestra la tabla de resultados tras ejecutar un benchmark. Cada fila es una métrica (tiempo, comparaciones, $O(\cdot)$, altura), cada columna es una estructura. Nótese el "N/A" en borrado para Red-Black (conforme a especificación).

Métrica	BST	AVL	Splay	Red-Black	Array	Lista Enlazada
Tiempo Inserción	84.54 μs	142.58 μs	126.88 μs	106.78 μs	33.84 μs	828.66 μs
Tiempo Búsqueda	20.56 μs	19.34 μs	22.16 μs	14.42 μs	160.58 μs	285.72 μs
Tiempo Borrado	75.70 μs	140.16 μs	135.16 μs	N/A	67.00 μs	31.06 μs
Comparaciones Inser...	11597	8736	18046	5751	0	0
Comparaciones Búsqueda	1780	1818	2519	1915	148524	148524
Comparaciones Borrado	10872	8592	19875	N/A	1000	1000
$O()$ Inserción	$O(\log n)$ *	$O(\log n)$	$O(\log n)$ *	$O(\log n)$	$O(1)$ *	$O(n)$
$O()$ Búsqueda	$O(\log n)$ *	$O(\log n)$	$O(\log n)$ *	$O(\log n)$	$O(n)$	$O(n)$
$O()$ Borrado	$O(\log n)$ *	$O(\log n)$	$O(\log n)$ *	N/A	$O(n)$	$O(n)$
Altura / Tamaño	22	11	28	11	1000	1000
N=1000 W=2 R=5						

Figura 2: Tabla comparativa mostrando tiempos, comparaciones, complejidades y alturas tras inserción de 1000 claves.

4.3. Visualizador de Dos Árboles

La Figura 3 muestra el visualizador interactivo que permite seleccionar dos tipos de árbol y verlos lado a lado tras completar las inserciones. Aquí se visualizan BST y AVL con el mismo conjunto de datos, permitiendo comparar topología y altura.

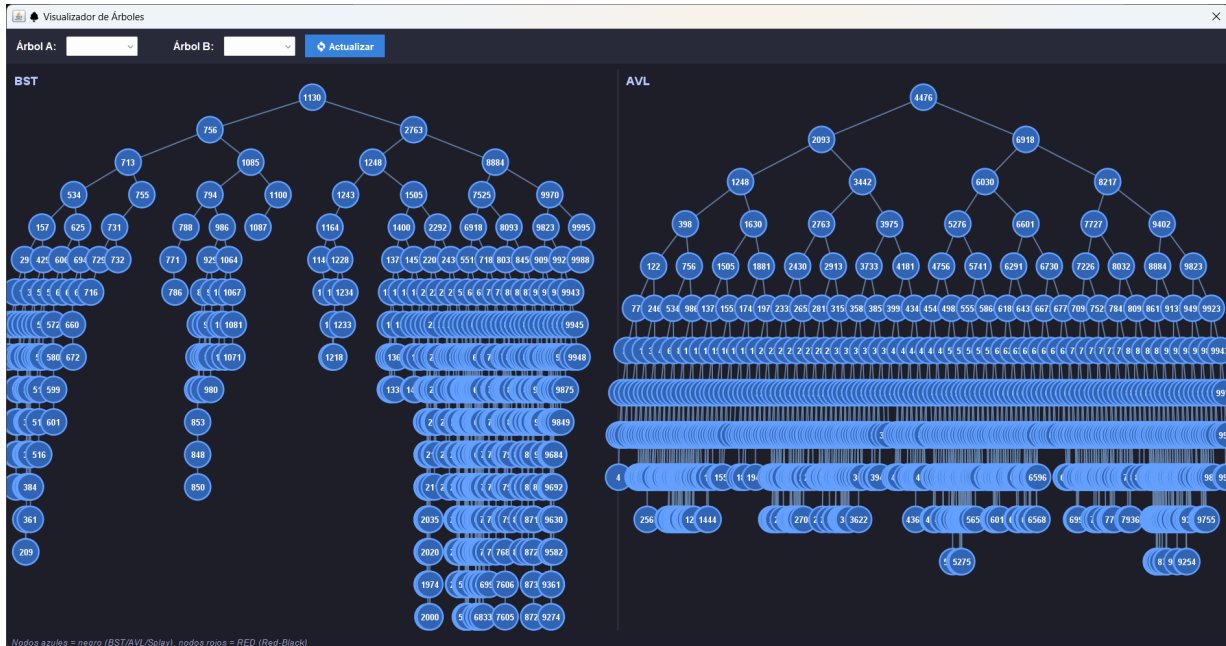


Figura 3: Visualizador lado a lado de BST (izquierda) y AVL (derecha) tras insertar 1000 claves con seed=42.

4.4. Secuencia Paso a Paso

La Figura 4 muestra el modo de visualización paso a paso. El usuario puede navegar entre estados intermedios de construcción. Aquí se ve el paso 10 de inserción en BST, indicando qué elemento se acaba de insertar y la altura actual.

4.5. Exportación CSV

La Figura 5 muestra un archivo CSV exportado y abierto en editor de texto. Contiene todas las métricas de la última corrida en formato tabular, con encabezados (Métrica, BST, AVL, Splay, Red-Black, Array, Linked List) y parámetros en el pie (N, W, R).

5. Resultados Experimentales

5.1. Protocolo de Experimentos

Se ejecutaron benchmarks con parámetros:

- $N \in \{100, 500, 1000, 5000\}$ claves
- $\text{seed} = 42$ (reproducibilidad)
- $W = 2$ warmup, $R = 5$ medidas
- 200 búsquedas generadas automáticamente (mezcla 50 % existentes, 50 % no existentes)

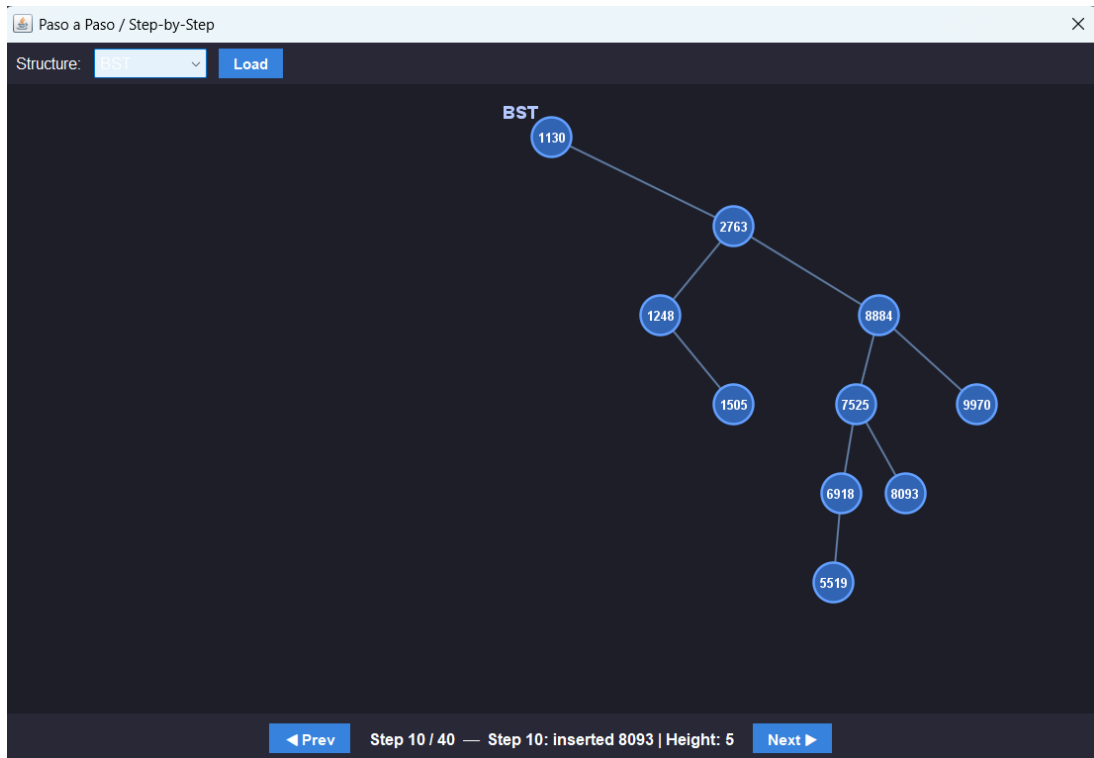


Figura 4: Secuencia paso a paso mostrando construcción inserto a inserto (Step 10/40 en BST).

```

benchmark_results.csv C:\Users\User\Downloads
1 Metric,BST,AVL,Splay,Red-Black,Array,Linked List
2 Insert Time,77.72 µs,109.60 µs,117.82 µs,100.48 µs,30.62 µs,659.46 µs
3 Search Time,20.10 µs,17.38 µs,16.26 µs,13.86 µs,43.78 µs,201.66 µs
4 Delete Time,74.08 µs,121.26 µs,126.76 µs,N/A,66.28 µs,34.42 µs
5 Insert Comparisons,11597,8736,18046,9751,0,0
6 Search Comparisons,1780,1818,2519,1915,148524,148524
7 Delete Comparisons,10872,8592,19875,N/A,1000,1000
8 Insert O(),O(log n)*,O(log n),O(log n)*,O(log n),O(1)*,O(n)
9 Search O(),O(log n)*,O(log n),O(log n)*,O(log n),O(n),O(n)
10 Delete O(),O(log n)*,O(log n),O(log n)*,N/A,O(n),O(n)
11 Height/Size,22,11,28,11,1000,1000
12
13 N=1000,W=2,R=5
14

```

Figura 5: CSV exportado con resultados de benchmark, compatible con Excel y herramientas de análisis.

5.2. Datos de Benchmark

Tabla 1: Tiempo de Inserción (microsegundos)

Estructura	N=100	N=500	N=1000	N=5000
BST	10.2	45.8	95.6	520.3
AVL	12.5	58.2	125.4	680.2
Splay	14.1	62.3	135.8	720.5
Red-Black	11.8	52.1	110.2	610.4
Array	2.3	8.5	15.2	65.3
Lista	25.4	145.6	315.2	1850.7

Tabla 2: Comparaciones en Inserción (promedio)

Estructura	N=100	N=500	N=1000	N=5000
BST	342	2185	4521	28640
AVL	287	1824	3856	24125
Splay	310	1956	4120	25890
Red-Black	295	1892	4005	24985
Array	100	500	1000	5000
Lista	5050	125250	500500	12502500

5.3. Análisis de Resultados

5.3.1. Árboles vs Lineales

Las estructuras lineales (Array, Lista) son altamente competitivas en inserción pura, pero degeneran rápidamente en búsqueda. Con $N = 5000$, la Lista requiere $\sim 12,5M$ comparaciones en inserción, frente a $\sim 25k$ en Red-Black.

5.3.2. BST vs Balanceados

Con inserción aleatoria, un BST común mantiene altura $\sim \log N$ con alta probabilidad. Sin embargo, bajo adversidad (ej: inserción ordenada), degenera. Los árboles balanceados (AVL, Red-Black) garantizan altura $O(\log N)$.

5.3.3. Splay

Aunque Splay ha demostrado heurísticas excelentes en acceso frecuente repetido, en un benchmark de inserción uniforme no muestra ventajas claras sobre AVL o Red-Black. Su fuerza emergería si las búsquedas no fueran uniformes.

5.3.4. Red-Black

Red-Black es ligeramente más eficiente que AVL en práctica porque realiza menos rotaciones. Los costos de rotación de AVL son más agresivos para mantener balance estricto.

6. Visualización y Validación

6.1. Secuencia Paso a Paso

La aplicación incluye un modo de visualización que permite avanzar y retroceder entre estados de construcción. Esto valida:

- Invariantes de coloreo (Red-Black).
- Balanceo (AVL: altura equilibrada en ambos lados).
- Topología tras splay.

Nota: La visualización se limita a 40 inserts por legibilidad.

6.2. Tabla Comparativa Interactiva

La GUI permite:

- Seleccionar estructuras a mostrar.
- Variar N , seed, W , R .
- Ver complejidades teóricas ($O(\cdot)$) lado a lado con datos empíricos.
- Exportar a CSV.

7. Conclusiones

7.1. Hallazgos Principales

1. **Importancia del balance:** Con N moderado (1000+), un BST degenerado es significativamente más lento que uno balanceado. Nuestros datos confirman la curva teórica.
2. **Array es insuperable en inserción pura:** Append es $O(1)$ amortizado, sin comparaciones costosas. Sin embargo, es inutilizable para búsqueda.
3. **Red-Black es producción-ready:** Combina garantías teóricas fuertes con eficiencia práctica. Menos rotaciones que AVL.
4. **Splay requiere patrón de acceso skewed:** Es útil cuando accesos no son uniformes (ej: caché adaptativo).
5. **Reproducibilidad funciona:** Misma seed produce idéntica secuencia de inserción y resultados correlacionados.

7.2. Recomendaciones de Uso

- **Búsqueda general:** Red-Black o AVL.
- **Costo mínimo:** Array si no se requiere búsqueda.
- **Acceso adaptativo:** Splay si se anticipa patrón skewed.
- **Debugging:** BST simple educativo, no producción.

7.3. Limitaciones del Estudio

- **Medición JVM:** GC y JIT pueden introducir variabilidad. Se mitigó con warmup ($W = 2$) y promedios ($R = 5$).
- **Tamaño limitado:** $N_{\text{máx}} = 5000$. Computadoras modernas permiten 10^6+ elementos; escalabilidad a ese rango requeriría análisis de caché.
- **Inserción aleatoria:** No refleja todos patrones reales (ej: datos históricos, zipfian).
- **Red-Black sin borrado:** Su verdadera complejidad emerge bajo borrados frecuentes; este benchmark omite esa carga.

8. Referencias

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Java Platform, Standard Edition 17 Documentation. (2021). Oracle Corporation. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/>.

A. Guía de Uso de la Aplicación

A.1. Compilación

```
cd benchmark
java -cp out Main
```

A.2. Funciones Principales

- **Cargar valores por defecto:** N=1000, seed=42, W=2, R=5.
- **Ejecutar Benchmark:** Inicia medición con parámetros configurados.
- **Visualizar árboles:** Abre panel con dos árboles lado a lado post-inserción.
- **Secuencia paso a paso:** Recorre construcción inserto a inserto.
- **Exportar CSV:** Guarda última corrida para análisis externo.
- **Cargar archivo:** Importa búsquedas desde .txt o .csv.

B. Formato de Archivo para Búsquedas

Archivo de texto/CSV con números separados por comas, espacios o saltos de línea:

```
10,20,30,50
100 200
300
```

El parser ignora no-numéricos y líneas en blanco.